Complexity-Dependent Failure of Multi-Agent Frameworks in Repository-Level Issue Resolution

Raihan 韩子杰 (L25020030)
Lab 5: Research Gap Statement and Research Question Proposal
October 2026

1 Background and state of the art

Large language models (LLMs) are now the dominant approach in automated program repair (APR). Zhang et al. review 189 studies on LLM-based APR published between 2020 and September 2025 and describe how the way LLMs are adapted for repair has moved from fine-tuning, to few- and zero-shot prompting, and most recently to agents that plan and refine repairs using tool feedback [1, §6.2.5]. The most demanding setting is repository-level issue resolution, in which a system receives an informal GitHub issue and must produce a patch for the whole codebase. SWE-bench [3] defines this task with 2,294 issues from 12 Python repositories, and the number of repository-level studies in the review rose from 7 in 2024 to 11 in 2025 [1, Table 8]. MAGIS [2] is one of these systems. It splits the work among four LLM agents: a Manager that turns the issue into file-level tasks, a Repository Custodian that ranks files with BM25 and filters them using LLM-written file summaries, Developers that first choose the line ranges to edit and then rewrite them, and a QA Engineer that reviews each diff. With GPT-4 as the base model, MAGIS resolves 13.94% of a 25% subset of SWE-bench, compared with 1.74% for GPT-4 alone, and 25.33% of SWE-bench Lite, compared with 18.00% for SWE-agent and 16.11% for AutoCodeRover [2, Tables 2 and 4].

2 The unresolved gap

The improvement is concentrated on small changes. The issues MAGIS resolves have reference patches that touch 1.04 files and 1.66 hunks on average and never more than two files. The issues for which it produces an applicable patch that still fails the tests have reference patches averaging 1.61 files and 3.72 hunks, with up to 18 files and 52 hunks [2, Table 5]. With the multi-agent design in place, the number of modified functions, hunks and changed lines are still significant negative predictors of resolution (logistic regression coefficients of -1.55, -0.12 and -0.57, $p < 0.05$) [2, Table 3]. The failed patches are also large: on unresolved issues they delete 327 lines on average, against 14.5 lines in the reference patches [2, Table 5]. Resolution rates differ by repository from close to 0% to about 40% [2, Fig. 13]. In addition, the main SWE-bench results were obtained with the files that need modification given to the system [2, §4.1], so they do not measure localization from the issue text alone. Zhang et al. list localization from informal reports, edits across several modules and reliable validation as the main difficulties of repository-level repair [1, §7.4]. We have not found a study that measures how much multi-agent issue-resolution frameworks degrade as the required change spans more files and hunks when localization is not given, which component (localization or review) accounts for the drop, or what each resolved issue costs.

3 Why existing approaches fail

We see three causes, each of which can be measured. First, localization in MAGIS begins with lexical BM25 ranking of the issue text and then judges candidate files one at a time [2, Alg. 1]. It does not use the import and call relations that connect the separate locations of a multi-hunk fix. The paper's own analysis shows that precision matters here. For Claude-2, raising file recall from 29.58 to 51.06 lowered the resolution rate from 1.96% to 1.22%, which the authors attribute to irrelevant context, while line-location coverage is a significant positive predictor of resolution (coefficient 0.5997, $p < 0.05$) [2, §2]. When an issue requires several locations, each missed location lowers the chance of a correct patch. Second, the QA Engineer accepts or rejects a diff on the basis of an LLM prompt alone and does not run tests [2, Alg. 3]. It adds between 1.57 and 3.31 percentage points to the resolution rate [2, Table 2]. Zhang et al. cite evidence that LLMs often fail to give reliable feedback on why code is wrong [1, §8.4], [5], and note that patches can pass a test suite and still be wrong [1, §8.3]. A review that runs no tests at all gives even weaker guarantees, which fits the observation that approved patches sometimes delete hundreds of lines. Third, results are reported as one resolution rate over benchmarks where most reference patches touch a single file, in the setting where the correct files are given, and MAGIS reports running time but not token or monetary cost per issue. The review calls for cost-aware evaluation and for controlling benchmark data leakage [1, §9.2].

4 Proposed approach and expected contribution

We propose an ablation study in which issues are grouped by the complexity of their reference patch, combined with two changes to the MAGIS design. The first replaces the Repository Custodian with one that adds a repository dependency graph (imports and call edges) to BM25, so that locations which must change together are retrieved together. The second gives the QA Engineer the ability to run the repository's existing tests and an LLM-generated test that reproduces the issue before it approves a diff, and to return any failures to the Developer. All variants use the same base model and settings, and OpenHands [4] is included as a general-purpose agent baseline. Experiments use SWE-bench Lite and SWE-bench Verified without giving the system the files to modify. Issues are placed in three groups by reference patch: one file and one hunk (B1), one file and two or more hunks (B2), and two or more files (B3). We measure resolution

rate, file recall at k, the line-coverage ratio defined by Tao et al. [2, Eq. 1], and tokens, cost in US dollars and running time per resolved issue. The study would give a resolution curve over change complexity for multi-agent issue resolution, separate the effect of localization from the effect of review, compare role-based and general-purpose agents at equal cost, and release the scripts and data needed to replicate it. We expect both changes to help most in group B3.

5 Research questions

Table 1 lists the research questions together with their independent and dependent variables. For all three questions the base model, temperature, iteration limit and benchmark split are held constant, and each configuration is run three times and reported as mean and range.

Table 1: Research questions and variables.

	Question	Independent variables	Dependent variables
RQ1	How much does the resolution rate of multi-agent issue-resolution frameworks fall as the required change becomes more complex, and how much of the reported performance depends on being given the files to modify?	Change complexity (group B1, B2 or B3; number of files and hunks in the reference patch); localization setting (files given or not given); framework (MAGIS-style or OpenHands)	Resolution rate; patch application rate; logistic regression coefficient of resolution on complexity
RQ2	Does dependency-aware localization find the right files and lines more often than BM25 with summary filtering, and does this raise the resolution rate for multi-file issues (B3)?	Localization method (BM25 only, MAGIS Custodian, dependency-aware Custodian); complexity group as moderator	File recall at k ($k = 1, 3, 5$); line-coverage ratio; resolution rate per group
RQ3	How does the review method affect the resolution rate, the rate at which incorrect patches are approved, and the cost of each resolved issue?	Review method (no review, LLM-only review, review that runs tests)	Resolution rate; false approval rate (approved patches that fail the benchmark tests); tokens, cost and running time per resolved issue

References

[1] Q. Zhang, C. Fang, Y. Xie, Y. Ma, W. Sun, Y. Yang, and Z. Chen. A systematic literature review on large language models for automated program repair. arXiv:2405.01466v4, 2026.

[2] W. Tao, Y. Zhou, Y. Wang, W. Zhang, H. Zhang, and Y. Cheng. MAGIS: LLM-based multi-agent framework for GitHub issue resolution. In *NeurIPS*, 2024. arXiv:2403.17927.

[3] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*, 2024.

[4] X. Wang et al. OpenHands: An open platform for AI software developers as generalist agents. In *ICLR*, 2025.

[5] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama. Is self-repair a silver bullet for code generation? In *ICLR*, 2024.